

Practical Week-3

Task 1: Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

```
lenovo@Manya:~$ mkdir practical3
lenovo@Manya:~$ cd practical3
lenovo@Manya:~/practical3$ nano process.c
lenovo@Manya:~/practical3$ gcc process.c -o process
lenovo@Manya:~/practical3$ ./process

Before fork():
PID = 875
PPID = 390

Parent Process:
PID = 875
PPID = 390
Parent is waiting for child...
Child Process:
PID = 876
PPID = 875
Child is running...
Child process is terminating.
Child process has terminated.
lenovo@Manya:~/practical3$ ps
  PID TTY          TIME CMD
  390 pts/0    00:00:00 bash
 1057 pts/0    00:00:00 ps
lenovo@Manya:~/practical3$ ps -f
  UID          PID    PPID  C STIME TTY          TIME CMD
akshith+      390      389  0 06:43 pts/0    00:00:00 -bash
akshith+     1069      390  0 06:55 pts/0    00:00:00 ps -f
lenovo@Manya:~/practical3$
```

```
lenovo@Manya
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");
    printf("PID = %d\n", getpid());
    printf("PPID = %d\n\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("Child Process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

        printf("Child is running...\n");
        sleep(5);
        printf("Child process is terminating.\n");
    }
    else
    {
        printf("Parent Process:\n");
        printf("PID = %d\n", getpid());
        printf("PPID = %d\n", getppid());

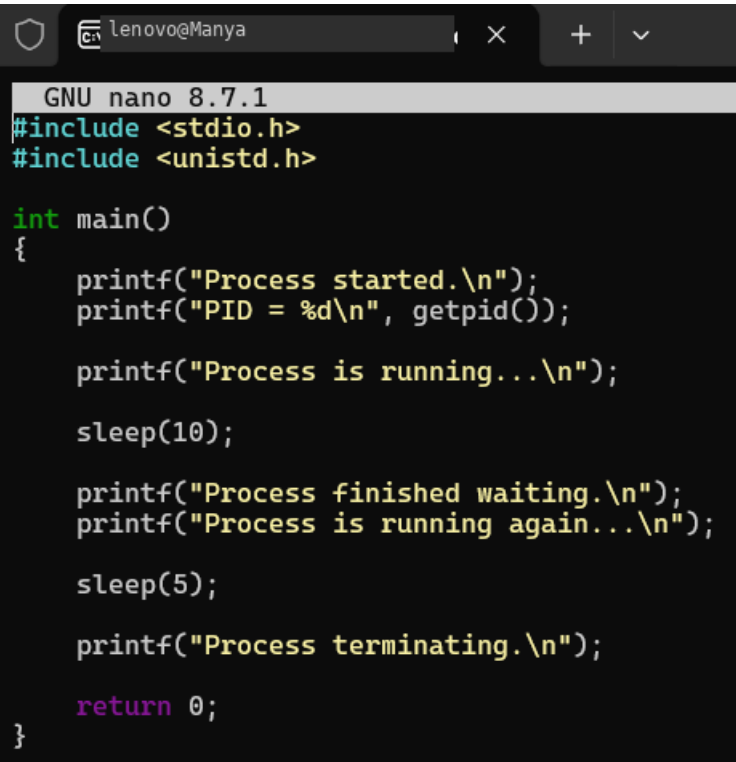
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Child process has terminated.\n");
    }

    return 0;
}
```

Task 2: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
lenovo@Manya:~/practical3$ mkdir task4
lenovo@Manya:~/practical3$ cd task4
lenovo@Manya:~/practical3/task4$ nano states.c
lenovo@Manya:~/practical3/task4$ gcc states.c -o states
lenovo@Manya:~/practical3/task4$ ./states
Process started.
PID = 1480
Process is running...
Process finished waiting.
Process is running again...
Process terminating.
lenovo@Manya:~/practical3/task4$ ps -ef | grep states
akshith+ 2064 390 0 08:12 pts/0 00:00:00 grep --color=auto states
lenovo@Manya:~/practical3/task4$ top
top - 08:12:47 up 35 min, 1 user, load average: 0.03, 0.07, 0.02
Tasks: 31 total, 1 running, 30 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.1 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 5769.6 total, 5187.7 free, 533.2 used, 192.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 5236.4 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    86 root        20   0   35600  12576  9268  S   0.7   0.2   0:07.41 systemd-udevd
     1 root        20   0   24200  15456 11628  S   0.0   0.3   0:01.01 systemd
     2 root        20   0    3180   2204  2072  S   0.0   0.0   0:00.01 init-systemd(Ub
     7 root        20   0    3180   2044  1940  S   0.0   0.0   0:00.00 init
    47 root        19  -1  50424  16740 15436  S   0.0   0.3   0:00.32 systemd-journal
    79 systemd+    20   0   22416  14336 11840  S   0.0   0.2   0:00.10 systemd-resolve
   153 root        20   0  159348   1880   1556  S   0.0   0.0   0:00.00 snapfuse
   154 root        20   0  454144  12172  1520  S   0.0   0.2   0:01.42 snapfuse
   158 root        20   0  159348   1856   1528  S   0.0   0.0   0:00.00 snapfuse
   184 root        20   0    2888   1948   1820  S   0.0   0.0   0:00.05 chronyd-starter
   185 root        20   0    4472   3056   2804  S   0.0   0.1   0:00.01 cron
   186 message+    20   0    8820   5396   4716  S   0.0   0.1   0:00.06 dbus-daemon
   192 root        20   0   44096  29632 14492  S   0.0   0.5   0:00.22 networkd-dispat
   199 root        20   0 2220940  40172 25040  S   0.0   0.7   0:02.10 snapd
   202 root        20   0   18436   9400   8156  S   0.0   0.2   0:00.09 systemd-logind
   220 syslog      20   0  220548   5880   4516  S   0.0   0.1   0:00.06 rsyslogd
   234 root        20   0    5492   2764   2580  S   0.0   0.0   0:00.01 agetty
   264 root        20   0  123460  32672 18064  S   0.0   0.6   0:00.18 unattended-upgr
   266 _chrony      20   0   23936  11424   7180  S   0.0   0.2   0:00.12 chronyd
   275 _chrony      20   0   12096   2340   1696  S   0.0   0.0   0:00.00 chronyd
   388 root        20   0    3188   1108    980  S   0.0   0.0   0:00.00 SessionLeader
   389 root        20   0    3204   1248   1108  S   0.0   0.0   0:00.10 Relay(390)
   390 akshith+    20   0    6268   5516   3848  S   0.0   0.1   0:00.12 bash
   391 root        20   0    8644   5496   4656  S   0.0   0.1   0:00.01 login
   434 akshith+    20   0   22768  13416 10948  S   0.0   0.2   0:00.13 systemd
   436 akshith+    20   0   22916   4100   2100  S   0.0   0.1   0:00.00 (sd-pam)
   464 akshith+    20   0    6136   5488   3928  S   0.0   0.1   0:00.03 bash
  1897 root        20   0 1792044  14280 11680  S   0.0   0.2   0:00.14 wsl-pro-service
  2082 akshith+    20   0    8192   6020   3900  R   0.0   0.1   0:00.00 top
```



A terminal window with a dark background. The title bar shows a shield icon, the username 'lenovo@Manya', and window controls. The terminal header is 'GNU nano 8.7.1'. The code is a C program for process management, using `printf` for output and `sleep` for delays. The program prints the PID, runs for 10 seconds, prints again, runs for 5 seconds, and then terminates.

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Process started.\n");
    printf("PID = %d\n", getpid());

    printf("Process is running...\n");

    sleep(10);

    printf("Process finished waiting.\n");
    printf("Process is running again...\n");

    sleep(5);

    printf("Process terminating.\n");

    return 0;
}
```